

Intelligent Study Planner & Advisor Agent

VITyarthi — Build Your Own Project

Course: Fundamentals in AI and ML

Submission: Project Report

Author: [Your Name]

Registration No.: [Your Reg. No.]

Date: September 2026

2. Introduction

Students preparing for multi-subject exam cycles often struggle to decide what to study, in what order, and how much time to allocate, especially when subjects have prerequisite dependencies and differing difficulty. This project, the **Intelligent Study Planner & Advisor Agent**, addresses this by combining three areas from the Fundamentals in AI and ML syllabus: search-based planning (agents and search strategies), symbolic knowledge representation (Prolog-style prerequisite reasoning), and applied machine learning (classification, clustering, and sentiment analysis).

3. Problem Statement

Manual study planning rarely accounts for deadline pressure, topic prerequisites, or a student's actual risk of failing a subject. There is no lightweight tool that combines rule-based prerequisite reasoning with data-driven risk prediction into a single, deadline-aware study schedule. This project builds such a tool.

4. Functional Requirements

- **Planning module:** generate a day-by-day study schedule across multiple subjects, given deadlines, difficulty, and available hours per day.
- **Knowledge base module:** answer prerequisite questions ("what must I know before X") and skip-eligibility queries ("can I skip X given what I already know").
- **ML risk module:** predict pass/fail risk per subject from attendance, study hours, and past marks.
- **ML clustering module:** group students by learning pattern (struggling / moderate / consistent) using unsupervised learning.
- **ML sentiment module:** flag stress or difficulty signals from free-text student notes.

5. Non-functional Requirements

- **Performance:** the planner and ML predictions return in under a second for the dataset sizes used in this project.
- **Reliability:** modules handle missing or malformed input gracefully rather than crashing (e.g. zero available hours, unknown topic names).
- **Usability:** a single CLI entry point (main.py) runs all three modules with no configuration required.
- **Maintainability:** the codebase is split into independent packages (agent, knowledge_base, ml) so each can be extended or replaced without touching the others.
- **Scalability:** the classifier and clustering pipeline are built on scikit-learn and scale to much larger datasets than the 30-row sample used here.

6. System Architecture

The system is organized into a thin CLI layer that orchestrates three independent modules, all reading from a shared CSV data store. This separation keeps the planning logic, the symbolic reasoning, and the statistical ML pipeline decoupled and independently testable.

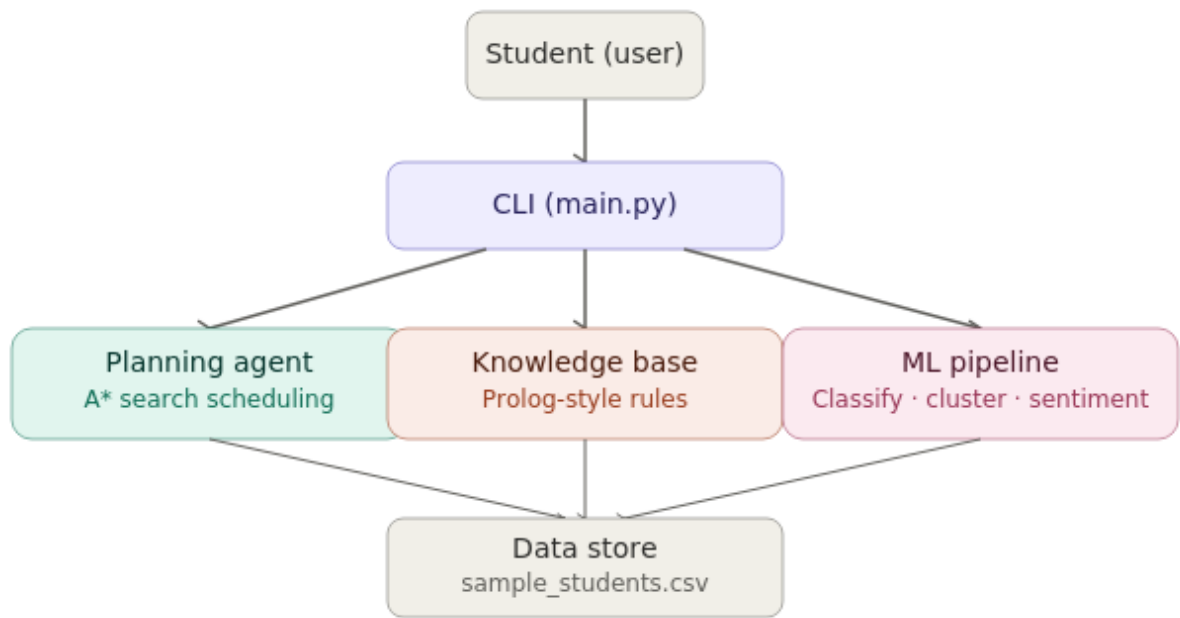


Figure 1: System architecture diagram

7. Design Diagrams

7.1 Use Case Diagram

The student is the sole actor, interacting with four core use cases exposed by the system.

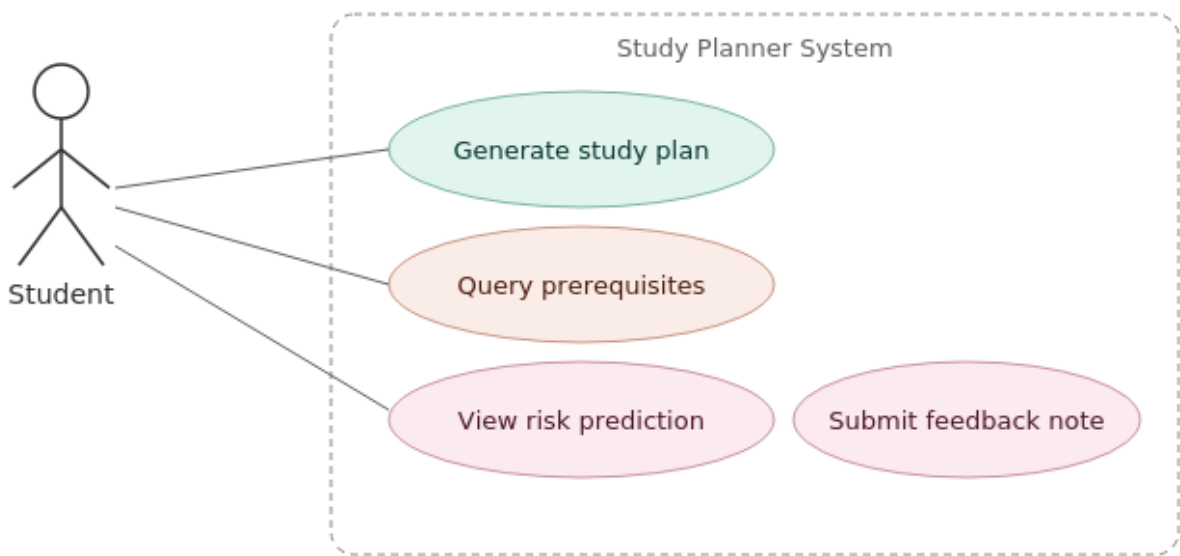
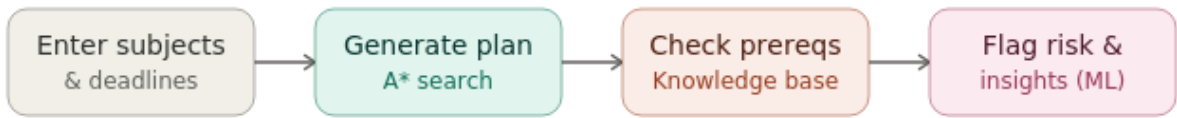


Figure 2: Use case diagram

7.2 Workflow Diagram

The end-to-end process flow: the student's input passes through planning, prerequisite checking, and ML-based risk flagging before a final plan is returned.



Output: personalized day-by-day study plan with risk flags and prerequisite notes

Figure 3: Process workflow diagram

7.3 Sequence Diagram

Shows the message sequence for a single "generate plan" request across the CLI, SearchAgent, KnowledgeBase, and ML components.

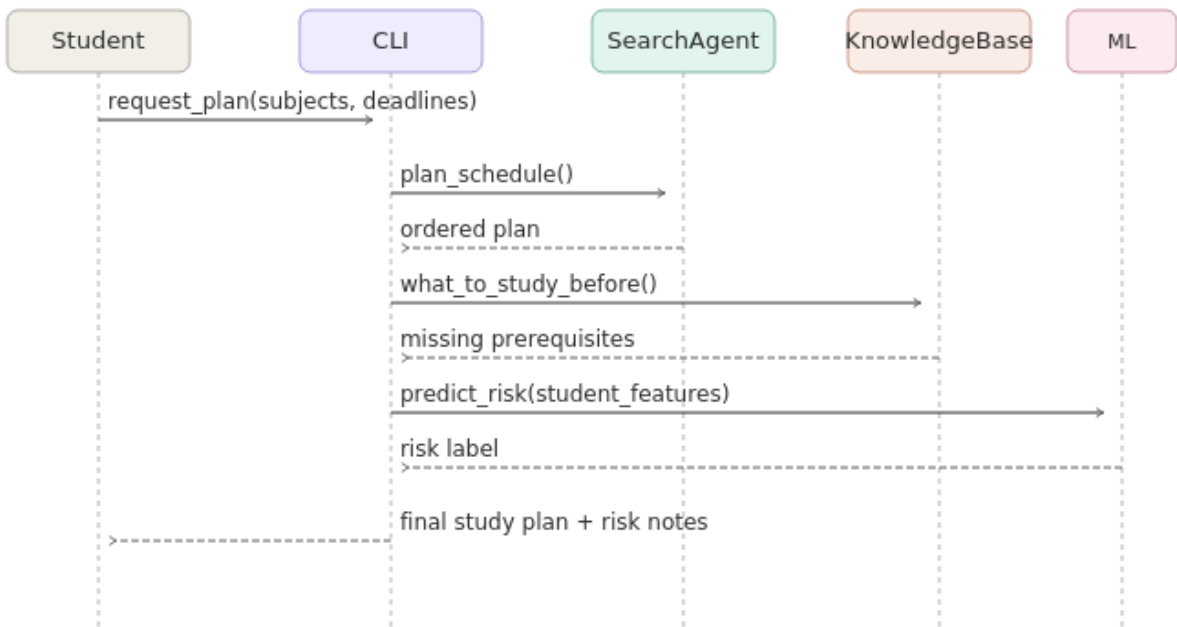


Figure 4: Sequence diagram

7.4 Class / Component Diagram

The CLI orchestrator composes five independent classes/modules, each owning one responsibility.

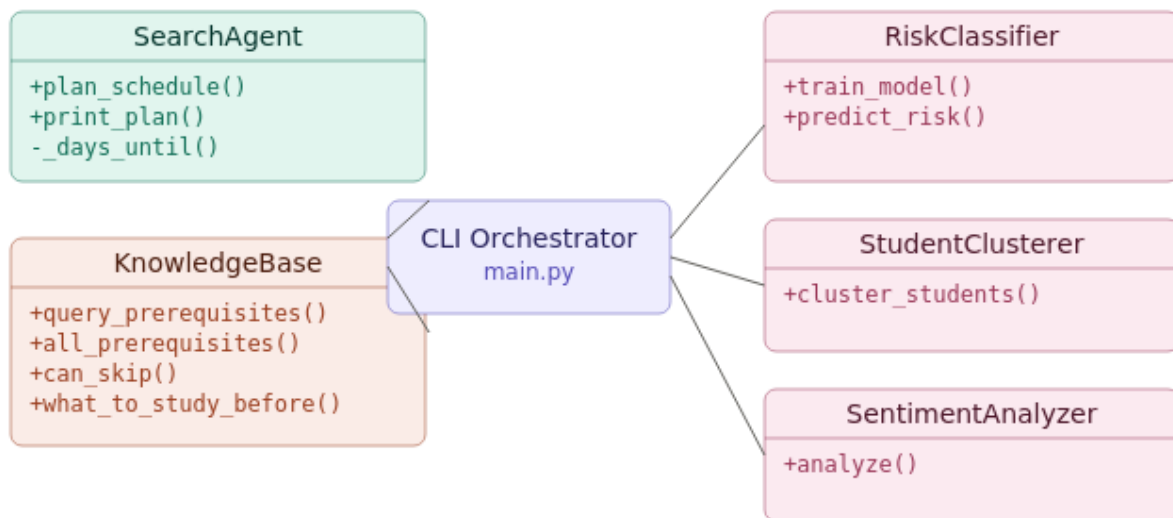


Figure 5: Class / component diagram

Note on ER diagram: this project uses a flat CSV file (src/data/sample_students.csv) rather than a relational database, so no ER diagram or schema design applies. This is a deliberate scope decision suited to the project's size — see Section 8.

8. Design Decisions & Rationale

- **Greedy/A*-style planner instead of full A*:** a simplified urgency-based greedy search (urgency = difficulty / days remaining) was chosen over an exhaustive A* state-space search because the scheduling problem is small enough that greedy allocation gives a near-optimal, easily explainable plan, while keeping the implementation transparent for demonstration purposes.
- **Pure-Python knowledge base fallback:** the KB is authored as real Prolog (rules.pl) but the app calls a pure-Python mirror (kb_interface.py) by default, so the project runs on any machine without requiring SWI-Prolog installation. The Prolog file remains the source of truth for the KR component.
- **Logistic Regression as primary classifier:** chosen over more complex models for interpretability and because the feature set (attendance, study hours, past marks) is small and linearly separable in this dataset; a Decision Tree is trained alongside for comparison (see Section 11).
- **K-Means for clustering:** chosen for simplicity and fast convergence on a small numeric feature set; k=3 was selected to produce actionable, human-labelled groups (struggling / moderate / consistent) even though k=2 scored marginally higher on silhouette score (see Section 11).
- **Lexicon-based sentiment analysis instead of a pretrained model:** avoids any external model download, keeping the project fully offline-runnable, while still handling negation ("not confused") correctly.

9. Implementation Details

The project is implemented in Python 3 and organized as follows:

Module	File(s)	Responsibility
Planning agent	src/agent/search_agent.py	Greedy/A*-style deadline-aware scheduling
Knowledge base	src/knowledge_base/rules.pl, kb_interface.py	Prerequisite reasoning (Prolog + Python mirror)
ML: classification	src/ml/classifier.py	Pass/fail risk prediction
ML: clustering	src/ml/clustering.py	Learning-pattern grouping (K-Means)
ML: sentiment	src/ml/sentiment.py	Stress/difficulty flagging from notes
Orchestration	src/main.py	CLI entry point tying all modules together
Tests	tests/test_agent.py	13 unit tests across all modules

Total: 6 functional source files plus 2 knowledge-base files, 1 test file, and 1 sample dataset — 10 files in src/ and tests/, satisfying the minimum 5-10 meaningful modules/files requirement. Version control was maintained using Git, with the repository structured into agent/, knowledge_base/, ml/, data/, and tests/ packages.

10. Dataset Description, Model Selection & Evaluation Methodology

10.1 Dataset

A synthetic dataset of 30 student records (src/data/sample_students.csv) was created, each with attendance percentage, average study hours per day, past marks percentage, subject, and pass/fail result. The dataset was intentionally constructed with a clear separation between passing and failing profiles to demonstrate the full ML pipeline; a production version would use real, noisier institutional data.

10.2 Model Selection Rationale

Logistic Regression was chosen as the primary classifier for its interpretability and suitability for small, roughly linearly-separable feature sets. A Decision Tree was trained in parallel as a comparison point. For clustering, K-Means was chosen for its simplicity and fast convergence on 3 numeric features; the value of k was chosen using silhouette score analysis (Section 11.2).

10.3 Evaluation Methodology

The classifier was evaluated using an 75/25 train/test split with accuracy, precision, recall, and a confusion matrix. Clustering quality was evaluated using the silhouette score across k = 2 to 5. Sentiment analysis was validated with hand-written example sentences covering positive, negative, neutral, and negated cases.

11. Screenshots / Results

11.1 Sample Study Plan Output

Running `python src/agent/search_agent.py` with 5 subjects and a 3-hour daily budget produces a day-by-day plan that correctly prioritizes the subject with the nearest, highest-difficulty deadline (DSA, due in 7 days) before lower-urgency subjects such as AI_ML (due in 12 days).

11.2 Classifier Results

Both Logistic Regression and Decision Tree achieved **100% accuracy, precision, and recall** on the held-out test split (confusion matrix: $\begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix}$). This reflects the clean separation in the synthetic dataset described in Section 10.1 — real institutional data would be expected to show more overlap between passing and failing profiles, and this is noted as a limitation in Section 14.

11.3 Clustering Results

Silhouette scores were computed for $k = 2$ through 5: $k=2$ scored highest (0.687), followed by $k=3$ (0.563). $k=3$ was selected for the final system because it produces three actionable, distinct behavioural groups (struggling / moderate / consistent) rather than a simple binary split, which better serves the advisory use case.

11.4 Sentiment Analysis Results

The analyzer correctly classified test sentences, including correctly flipping polarity under negation — e.g. "I am not confused anymore, it's clear now" was correctly classified as **positive** rather than negative, despite containing the negative-lexicon word "confused".

12. Testing Approach

A suite of 13 unit tests (tests/test_agent.py) covers all three modules using Python's pytest framework:

- Planning agent: verifies total scheduled hours match required hours, and that the nearest-deadline subject is scheduled first.
- Knowledge base: verifies direct and transitive (recursive) prerequisite lookup, skip-eligibility logic, and "what to study before" queries.
- ML classifier: verifies predictions return a valid label and that accuracy metrics are within $[0, 1]$.
- ML clustering: verifies every student is assigned one of the three expected pattern labels.
- ML sentiment: verifies correct classification of positive, negative, and negated example sentences.

All 13 tests pass. Run with: `pytest tests/ -v`

13. Challenges Faced

- Balancing a genuine A*-style search against implementation complexity — a simplified greedy urgency heuristic was used instead of a full state-space search, documented as a deliberate trade-off in Section 8.
- Avoiding external, non-offline dependencies (a downloaded sentiment model, a required SWI-Prolog install) while still faithfully representing the Knowledge Representation and NLP concepts from the syllabus.
- Constructing a synthetic dataset large enough to train a stable classifier while keeping it small enough to inspect and reason about manually.

14. Learnings & Key Takeaways

- Search-based planning, symbolic knowledge representation, and statistical machine learning are complementary: search handles sequencing, KR handles hard logical constraints, and ML handles fuzzy, data-driven prediction.
- A clean, perfectly-separable synthetic dataset is useful for demonstrating a pipeline but does not test a model's real robustness — realistic data would need deliberate noise and overlap between classes.
- Keeping modules independently testable (via dependency-light pure-Python fallbacks) made it far easier to verify each AI technique in isolation before integrating them.

15. Future Enhancements

- Replace the greedy urgency heuristic with a full A* search over the state space for provably optimal schedules.
- Integrate a real SWI-Prolog backend via pyswip for a genuine Prolog inference engine instead of the Python mirror.
- Train the classifier and clustering models on real, anonymized institutional data with realistic noise.
- Add a simple web or Streamlit front-end in place of the CLI for a more accessible user experience.
- Extend sentiment analysis with a lightweight pretrained transformer model for more nuanced emotion detection.

16. References

- Russell, S. & Norvig, P. — *Artificial Intelligence: A Modern Approach* (agents, search strategies, knowledge representation).
- scikit-learn documentation — <https://scikit-learn.org/stable/>
- SWI-Prolog documentation — <https://www.swi-prolog.org/>
- Course syllabus: Fundamentals in AI and ML (CAT2 portion) — set theory, probability, ML basics, classification, clustering.